

Algoritmi Avansati

Modele de examen rezolvate in detaliu

Partea I: aproximativi, TSP, genetici, probabilistici

Partea II: algoritmi geometrici

Cum se foloseste materialul. Pentru fiecare grila am pus raspunsul corect, explicatia si ideea de teorie care trebuie retinuta. Pentru intrebarile deschise am scris rezolvari complete, cu formulele care trebuie aplicate la examen. La finalul fiecarei parti exista o lista scurta cu lucruri care se pot cere ca teorie.

Contents

Surse si observatii	2
I Model Subiecte Part 1 - Rezolvare detaliata	3
1 Tabel rapid cu raspunsuri	3
2 Grile rezolvate si explicate	3
3 Intrebari deschise - rezolvari detaliata	7
4 Ce ar mai putea sa dea ca teorie la Partea I	9
II Model Subiecte Part 2 - Algoritmi Geometrici	10
5 Tabel rapid cu raspunsuri	10
6 Grile rezolvate si explicate	10
7 Intrebari deschise - rezolvari detaliata	14
8 Ce ar mai putea sa dea ca teorie la Partea II	19

Surse si observatii

Acest document rezolva modelele incarcate: **Model Subiecte Part1 3.pdf** si **Model Subiecte Part2__Alg_Geometrici.pdf**. Am pastrat structura de examen: grile cu un singur raspuns corect, apoi intrebari deschise cu justificare.

Atentie la examen: ai cursurile pe masa, dar timpul este scurt. La grile trebuie sa recunosti rapid formula / proprietatea. La intrebarile deschise nu scrie povesti lungi: scrie definitia, formula, apoi concluzia.

Part I

Model Subiecte Part 1 - Rezolvare detaliata

1 Tabel rapid cu raspunsuri

Intrebarea	1	2	3	4	5	6	7	8
Raspuns	b	a	b	c	c	b	b	b

2 Grile rezolvate si explicate

1. Reduceri polinomiale

Enunt pe scurt. Daca $A \leq_P B$ si $B \in P$, ce putem afirma despre A ?

Raspuns corect: (b) Problema A apartine clasei P .

Explicatie. Relatia

$$A \leq_P B$$

inseamna ca orice instanta a problemei A poate fi transformata in timp polinomial intr-o instanta a problemei B , astfel incat raspunsul se pastreaza. Daca B se rezolva in timp polinomial, atunci pentru a rezolva A procedam asa:

1. transformam instanta lui A intr-o instanta a lui B in timp polinomial;
2. rezolvam instanta lui B in timp polinomial;
3. interpretam raspunsul inapoi pentru A .

Suma a doua timpuri polinomiale este tot polinomiala. Deci $A \in P$.

De retinut: daca $A \leq_P B$, atunci A nu este mai grea decat B . Daca stii sa rezolvi eficient B , stii sa rezolvi eficient si A .

2. Load Balancing - LPT si joburi puteri ale lui 2

Enunt pe scurt. LPT sorteaza joburile descrescator. Pentru joburi puteri ale lui 2, de exemplu $\{1, 2, 4, 8\}$, si doua masini, ce putem spune despre solutia LPT?

Raspuns corect: (a) Va fi intotdeauna optima, pentru cazul din intrebare.

Explicatie pe exemplu. Joburile sortate descrescator sunt:

$$8, 4, 2, 1.$$

Aplicam LPT pe doua masini:

- punem 8 pe M_1 : load-uri $(8, 0)$;
- punem 4 pe M_2 : load-uri $(8, 4)$;
- punem 2 pe M_2 : load-uri $(8, 6)$;
- punem 1 pe M_2 : load-uri $(8, 7)$.

Deci

$$ALG = 8.$$

Suma totala este:

$$1 + 2 + 4 + 8 = 15.$$

Orice solutie pe doua masini are makespan cel putin media:

$$\text{OPT} \geq \left\lceil \frac{15}{2} \right\rceil = 8.$$

Dar LPT a obtinut 8, deci:

$$\text{ALG} = \text{OPT} = 8.$$

Idee generala pentru examen. LPT este bun deoarece pune intai joburile mari. Pentru doua masini si structuri foarte ordonate, cum sunt puterile lui 2 din exemplu, greedy echilibreaza incarcarea cat permite suma totala.

Lower bound obligatoriu: in Load Balancing folosesti aproape mereu

$$\text{OPT} \geq \max \left\{ \frac{1}{m} \sum_j t_j, \max_j t_j \right\}.$$

3. Weighted Vertex Cover si pragul 1/2

Enunt pe scurt. In algoritmul pentru Weighted Vertex Cover bazat pe programare liniara, de ce rotunjim variabilele $x_i \in [0, 1]$ cu pragul 1/2?

Raspuns corect: (b) Pentru a garanta ca fiecare muchie este acoperita.

Modelul LP. Pentru fiecare varf v avem o variabila $x_v \in [0, 1]$. Intuitiv:

$$x_v = 1 \Rightarrow v \text{ este ales}, \quad x_v = 0 \Rightarrow v \text{ nu este ales}.$$

Programul liniar relaxat este:

$$\min \sum_{v \in V} w(v)x_v$$

cu restrictiile:

$$\begin{aligned} x_u + x_v &\geq 1, & \forall (u, v) \in E, \\ 0 \leq x_v &\leq 1, & \forall v \in V. \end{aligned}$$

Rotunjire. Alegem:

$$S = \{v \in V \mid x_v \geq 1/2\}.$$

Pentru orice muchie (u, v) , LP-ul impune:

$$x_u + x_v \geq 1.$$

Daca am avea simultan $x_u < 1/2$ si $x_v < 1/2$, atunci:

$$x_u + x_v < 1,$$

contradictie. Deci cel putin unul dintre x_u, x_v este $\geq 1/2$, deci cel putin unul dintre capetele muchiei este selectat. Astfel S este vertex cover valid.

Factor de aproximare. Daca $x_v \geq 1/2$, atunci $1 \leq 2x_v$. Deci:

$$w(S) = \sum_{v \in S} w(v) \leq \sum_{v \in S} 2w(v)x_v \leq 2 \sum_{v \in V} w(v)x_v \leq 2\text{OPT}.$$

Algoritmul este 2-aproximativ.

4. Algoritmi genetici - elitism agresiv

Enunt pe scurt. Care este riscul major al unui elitism prea agresiv?

Raspuns corect: (c) Convergenta rapida catre un optim local si pierderea diversitatii genetice.

Explicatie. Elitismul inseamna ca cei mai buni indivizi trec direct in generatia urmatoare. Este util fiindca nu pierdem cea mai buna solutie gasita pana acum. Dar daca pastram prea multi indivizi elitisti, populatia devine foarte asemanatoare foarte repede.

Consecinta:

- diversitatea scade;
- crossover-ul combina indivizi aproape identici;
- mutatia poate sa nu mai fie suficienta pentru explorare;
- algoritmul ramane blocat intr-un optim local.

De retinut: elitismul imbunatateste stabilitatea, dar prea mult elitism omoara explorarea.

5. Freivalds cu vector aleator din $\{0, 1, \dots, 99\}$

Enunt pe scurt. In Freivalds, daca alegem r_i din $\{0, 1, \dots, 99\}$ in loc de $\{0, 1\}$, ce se intampla cu probabilitatea de eroare?

Raspuns corect: (c) Scade la cel mult 1/100.

Algoritmul Freivalds. Vrem sa verificam daca:

$$AB = C.$$

In loc sa calculam direct AB in $\mathcal{O}(n^3)$, alegem un vector aleator r si verificam:

$$A(Br) = Cr.$$

Costul este $\mathcal{O}(n^2)$.

De ce exista eroare? Daca $AB = C$, atunci:

$$A(Br) = (AB)r = Cr,$$

deci algoritmul raspunde mereu corect cu DA.

Daca $AB \neq C$, notam:

$$D = AB - C.$$

Atunci $D \neq 0$. Freivalds greseste doar daca:

$$Dr = 0.$$

Daca alegem coordonatele lui r uniform dintr-o multime cu q valori, probabilitatea ca o ecuatie liniara nenula sa fie satisfacuta accidental este cel mult:

$$\frac{1}{q}.$$

Aici $q = 100$, deci eroarea este:

$$\mathbb{P}[\text{eroare}] \leq \frac{1}{100}.$$

Pentru vector binar $\{0, 1\}$, limita clasica este 1/2. Pentru 100 valori, limita devine 1/100.

6. Randomized Quicksort - pivot bun

Enunt pe scurt. Pivotul este bun daca ambele subliste au cel putin $1/4$ din elemente. Care este probabilitatea?

Raspuns corect: (b) $1/2$.

Explicatie. Daca sortam conceptual elementele, pivotul este bun daca nu este in primul sfert si nici in ultimul sfert. Adica pivotul trebuie sa fie in jumatatea centrala:

$$\left[\frac{n}{4}, \frac{3n}{4} \right].$$

Numarul de pozitii bune este aproximativ:

$$\frac{n}{2}.$$

Cum pivotul este ales uniform aleatoriu dintre cele n pozitii:

$$\mathbb{P}[\text{pivot bun}] = \frac{n/2}{n} = \frac{1}{2}.$$

Paranoid Quicksort: alege pivoti pana gaseste unul bun. Daca probabilitatea este $1/2$, numarul asteptat de incercari este 2.

7. Skip List - numarul asteptat de pointeri

Enunt pe scurt. Skip List cu n elemente si probabilitate de promovare $p = 1/2$. Numar total asteptat de pointeri?

Raspuns corect: (b) $2n$.

Explicatie. Un element apare sigur pe nivelul 0. Apoi este promovat pe nivelul 1 cu probabilitate $1/2$, pe nivelul 2 cu probabilitate $(1/2)^2$, etc.

Numarul asteptat de niveluri pe care apare un element este:

$$1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots = \frac{1}{1 - 1/2} = 2.$$

Daca fiecare aparitie contribuie cu un pointer orizontal, atunci numarul total asteptat de pointeri este:

$$2n.$$

8. Bloom Filter saturat

Enunt pe scurt. Daca toti bitii din Bloom Filter sunt 1, ce raspunde $Query(x)$?

Raspuns corect: (b) Returneaza intotdeauna DA, generand fals-pozitiv pentru orice element care nu e in multime.

Explicatie. $Query(x)$ calculeaza cele k pozitii date de functiile de hash si verifica daca toti bitii sunt 1. Daca tot vectorul este plin cu 1, atunci orice element va parea prezent:

$$\forall x, \quad Query(x) = DA.$$

Pentru elementele care nu au fost inserate, acest DA este fals-pozitiv.

Bloom Filter nu are fals-negative: daca raspunde NU, elementul sigur nu este in multime. Daca raspunde DA, poate fi in multime sau poate fi fals-pozitiv.

3 Intrebări deschise - rezolvari detaliate

9. Metric TSP - de ce adaugarea unui nod nu poate scadea OPT

Cerinta. Demonstrati ca daca adaugam un nod nou intr-o instanta de Metric TSP, costul turului optim nu poate scadea.

Notatii. Fie V multimea initiala de noduri si fie w nodul nou. Notam:

$$\text{OPT}(V) = \text{costul optim pe nodurile initiale},$$

$$\text{OPT}(V \cup \{w\}) = \text{costul optim dupa adaugarea lui } w.$$

Vrem sa aratam:

$$\text{OPT}(V) \leq \text{OPT}(V \cup \{w\}).$$

Asta inseamna ca adaugarea unui nod nu poate face solutia mai ieftina.

Demonstratie. Luam turul optim pentru $V \cup \{w\}$. In acest tur, nodul w are doi vecini, sa zicem u si v :

$$u \rightarrow w \rightarrow v.$$

Eliminam nodul w si unim direct u cu v . Obtinem un tur valid pentru nodurile din V .

Prin inegalitatea triunghiului:

$$d(u, v) \leq d(u, w) + d(w, v).$$

Deci, dupa eliminarea lui w , costul turului nu creste, ci scade sau ramane egal.

Daca turul initial avea cost $\text{OPT}(V \cup \{w\})$, turul obtinut pe V are cost cel mult:

$$\text{OPT}(V \cup \{w\}).$$

Dar $\text{OPT}(V)$ este cel mai mic cost posibil pentru V , deci:

$$\text{OPT}(V) \leq \text{OPT}(V \cup \{w\}).$$

Concluzie. Costul optim nu poate scadea cand adaugam un nod nou.

10. Bloom Filter - structura probabilistica cu eroare unilaterala

Cerinta. De ce Bloom Filter are eroare unilaterala? Care raspuns poarta incertitudinea?

Raspuns. Bloom Filter este probabilistic deoarece foloseste functii de hash si poate produce coliziuni. Eroarea este unilaterala deoarece doar raspunsul DA poate fi gresit. Raspunsul NU este sigur.

De ce NU este sigur? Cand inseram un element, toate pozitiile sale hash sunt setate la 1. Daca la $\text{Query}(x)$ gasim macar o pozitie cu bit 0, atunci x nu putea fi inserat, fiindca inserarea lui ar fi setat acel bit la 1. Deci:

$$\text{Query}(x) = \text{NU} \Rightarrow x \notin S.$$

De ce DA poate fi gresit? Daca toate cele k pozitii sunt 1, exista doua posibilitati:

- x a fost inserat;
- x nu a fost inserat, dar acei biti au fost setati de alte elemente.

Al doilea caz este fals-pozitiv.

$$\text{Query}(x) = \text{DA} \Rightarrow x \in S \text{ sau fals-pozitiv.}$$

11. TSP general - de ce nu admite aproximare constanta

Cerinta. Explicati reducerea de la Ciclu Hamiltonian.

Ideea. Presupunem prin absurd ca exista un algoritm α -aproximativ pentru TSP general, unde α este o constanta. Vom arata ca am putea decide Hamiltonian Cycle in timp polinomial.

Constructie. Fie $G = (V, E)$ un graf neorientat cu n noduri. Construim graful complet G' pe aceleasi noduri. Definim costurile:

$$c(u, v) = \begin{cases} 1, & (u, v) \in E, \\ M, & (u, v) \notin E, \end{cases}$$

unde alegem M foarte mare, de exemplu:

$$M = \alpha n + 1.$$

Cazul 1: G are ciclu Hamiltonian. Atunci exista un tur care foloseste doar muchii din E , fiecare de cost 1. Deci:

$$\text{OPT}(G') = n.$$

Algoritmul α -aproximativ ar returna un tur de cost:

$$\text{ALG} \leq \alpha \text{OPT} = \alpha n.$$

Cazul 2: G nu are ciclu Hamiltonian. Orice tur Hamiltonian in graful complet G' trebuie sa foloseasca macar o muchie care nu era in G , deci macar o muchie de cost M . Atunci costul oricarui tur este:

$$\text{ALG} \geq M = \alpha n + 1 > \alpha n.$$

Decizie. Rulam algoritmul de aproximare pentru G' :

- daca returneaza cost $\leq \alpha n$, atunci G are ciclu Hamiltonian;
- daca returneaza cost $> \alpha n$, atunci G nu are ciclu Hamiltonian.

Am rezolva HC in timp polinomial, contradictie cu $P \neq NP$.

Concluzie. TSP general nu admite algoritm de aproximare cu factor constant, presupunand $P \neq NP$.

12. Las Vegas vs Monte Carlo

Cerinta. Diferenta fundamentala privind corectitudinea si timpul.

Tip algoritm	Corectitudine	Timp
Las Vegas	raspunsul este intotdeauna corect	timpul este variabila aleatorie
Monte Carlo	raspunsul poate fi gresit cu probabilitate controlata	timpul este de obicei fix / garantat

Exemple.

- Randomized Quicksort este de tip Las Vegas: sorteaza corect mereu, dar timpul depinde de pivotii alesi.
- Freivalds este de tip Monte Carlo: ruleaza rapid, dar daca $AB \neq C$ poate raspunde gresit DA cu probabilitate mica.

Formula scurta pentru examen: Las Vegas = corect mereu, timp aleator. Monte Carlo = timp controlat, corect probabil.

4 Ce ar mai putea sa dea ca teorie la Partea I

1. Definitia reducerii polinomiale si sensul relatiei $A \leq_P B$.
2. Diferenta dintre P , NP , NP-hard si NP-complet.
3. Demonstratia pentru Load Balancing: lower bound-uri si factorul $2 - 1/m$.
4. Diferenta dintre List Scheduling si LPT.
5. TSP general vs Metric TSP.
6. De ce TSP general nu are aproximare constanta.
7. Algoritmul MST pentru Metric TSP si factorul 2.
8. Christofides: MST + matching pe varfuri impare + ciclu eulerian + shortcut, factor $3/2$.
9. Vertex Cover simplu: alegem o muchie, punem ambele capete, stergem incidentele; factor 2.
10. Weighted Vertex Cover prin LP si prag $1/2$.
11. Algoritmi genetici: selectie, crossover, mutatie, elitism, convergenta prematura.
12. Freivalds: algoritmul, complexitate $\mathcal{O}(n^2)$, probabilitate de eroare.
13. Monte Carlo vs Las Vegas.
14. Skip List: inaltime asteptata $\mathcal{O}(\log n)$, spatiu asteptat $\mathcal{O}(n)$.
15. Bloom Filter: fals-pozitive, fara fals-negative.

Part II

Model Subiecte Part 2 - Algoritmi Geometrici

5 Tabel rapid cu raspunsuri

Intrebarea	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Raspuns	a	c	b	b	d	a	b	c	d	b	d	a	b	a	c	d

6 Grile rezolvate si explicate

1. Raportul $r(A, P, B) = 1$

Date: $A = (1, 2, 3)$, $B = (4, 5, 6)$.

Raspuns corect: (a) $P = \left(\frac{5}{2}, \frac{7}{2}, \frac{9}{2}\right)$.

Explicatie. Conditia $r(A, P, B) = 1$ inseamna:

$$\overrightarrow{AP} = \overrightarrow{PB}.$$

Deci P este mijlocul segmentului AB :

$$P = \frac{A+B}{2} = \left(\frac{1+4}{2}, \frac{2+5}{2}, \frac{3+6}{2}\right) = \left(\frac{5}{2}, \frac{7}{2}, \frac{9}{2}\right).$$

2. Graham vs Jarvis

Raspuns corect: (c) (i)-A, (ii)-F, (iii)-A, (iv)-A.

Explicatii.

- Graham's scan are complexitatea $\mathcal{O}(n \log n)$, fiindca sorteaza punctele.
- Jarvis' march nu este algoritm de baleiere orizontala, ci gift wrapping.
- Graham's scan se bazeaza pe sortare.
- Jarvis are complexitatea $\mathcal{O}(nh)$, unde h este numarul de puncte de pe acoperirea convexa. Daca h este mic, poate fi mai bun decat $\mathcal{O}(n \log n)$.

3. Jarvis' march - succesorul lui P_1

Raspuns corect: (b) 3 viraje la dreapta.

Explicatie. Punctele sunt:

$$P_1 = (0, 0), P_2 = (1, 2), P_3 = (2, 1), P_4 = (3, 0), P_5 = (5, 0), P_6 = (2, -3), P_7 = (5, -2), P_8 = (3.5, -2).$$

Pornim cu pivot P_2 . Testam punctele in ordine. Pivotul se schimba cand punctul testat este in dreapta muchiei orientate P_1S .

Conform testelor:

- P_3 este in dreapta lui P_1P_2 - schimbam pivotul in P_3 ;
- P_4 este in dreapta lui P_1P_3 - schimbam pivotul in P_4 ;

- P_5 nu este in dreapta lui P_1P_4 ;
- P_6 este in dreapta lui P_1P_4 - schimbam pivotul in P_6 ;
- P_7, P_8 nu sunt in dreapta lui P_1P_6 .

Au aparut 3 viraje la dreapta.

4. Clasificarea varfurilor convexe / concave

Raspuns corect: (b) $\mathcal{O}(n)$.

Explicatie. Pentru fiecare varf P_i verificam orientarea tripletului:

$$(P_{i-1}, P_i, P_{i+1}).$$

Testul este constant, deci pentru n varfuri:

$$T(n) = \mathcal{O}(n).$$

5. Graful dual al unei triangulări

Raspuns corect: (d) Pentru a justifica existenta unei 3-colorari.

Explicatie. In demonstratia teoremei galeriei de arta, triangulam poligonul. Graful dual al triangulării are:

- un nod pentru fiecare triunghi;
- o muchie intre doua noduri daca triunghiurile corespunzatoare au o latura comuna.

Pentru o triangulare a unui poligon simplu, graful dual este arbore. Acest lucru ajuta la colorarea inductiva cu trei culori.

6. Triangularea poligonului y-monoton

Raspuns corect: (a) 1: G , 2a: E , 2b: B, C, D .

Explicatie. In algoritmul de triangulare a unui poligon y-monoton, varfurile sunt procesate descrescator dupa coordonata y . Cazurile apar in functie de:

- daca varful curent este pe lant opus fata de varful din varful stivei;
- daca este pe acelasi lant si produce diagonale valide;
- tipul de viraj produs.

Pentru poligonul dat, asocierea corecta din barem este:

$$1 : G, \quad 2a : E, \quad 2b : B, C, D.$$

7. Triangularea multimii cu parametrul α

Raspuns corect: (b) $\alpha \in (2, 8)$.

Explicatie. Formula pentru o triangulare a unei multimii de n puncte, cu k puncte pe frontiera acoperirii convexe, este:

$$\#\text{triunghiuri} = 2n - k - 2,$$

$$\#\text{muchii} = 3n - k - 3.$$

Pentru $\alpha \in (2, 8)$, punctul $P_6 = (\alpha - 1, 5)$ este in interiorul pentagonului determinat de celelalte puncte. Avem:

$$n = 6, \quad k = 5.$$

Deci:

$$2n - k - 2 = 12 - 5 - 2 = 5,$$

$$3n - k - 3 = 18 - 5 - 3 = 10.$$

8. Compararea triangulărilor

Raspuns corect: (c) Unghiurile triunghiurilor.

Explicatie. Triangularea Delaunay maximizeaza unghiul minim, in sens lexicografic al vectorului de unghiuri sortat crescator. De aceea, cand comparam triangulări, ne uitam la unghiuri, nu la arii sau mediane.

9. Fortune - localizarea frunzei in statutul T

Raspuns corect: (d) $\mathcal{O}(\log n)$.

Explicatie. In algoritmul lui Fortune, statutul T este mentinut ca arbore de cautare echilibrat pentru arcele de pe beach line. Localizarea arcului afectat de un eveniment de tip sit se face prin cautare in acest arbore:

$$\mathcal{O}(\log n).$$

10. Numarul de semidrepte in diagrama Voronoi

Raspuns corect: (b) 15 puncte, exact 5 pe frontiera acoperirii convexe.

Regula importanta.

$$\# \text{semidrepte Voronoi} = \# \text{puncte de pe frontiera acoperirii convexe}.$$

Deci pentru exact 5 semidrepte trebuie exact 5 situri pe frontiera acoperirii convexe.

11. Cinci puncte si cinci semidrepte Voronoi

Raspuns corect: (d) $\{1, 2, 3\}$.

Explicatie. Daca diagrama Voronoi are cinci semidrepte si sunt cinci puncte, atunci toate cele cinci puncte sunt pe frontiera acoperirii convexe.

Numarul de varfuri poate fi:

- 1, daca toate cele cinci puncte sunt conciclice;
- 2, daca patru sunt conciclice si al cincilea nu este pe acelasi cerc;
- 3, daca nu exista patru puncte conciclice.

Deci multimea valorilor posibile este:

$$\{1, 2, 3\}.$$

12. Complexitatea algebrica: paralelogram vs dreptunghi

Raspuns corect: (a) paralelogram: grad I, dreptunghi: grad II.

Paralelogram. Un patrulater $ABCD$ este paralelogram daca diagonalele au acelasi mijloc:

$$A + C = B + D.$$

Pe coordonate:

$$x_A + x_C = x_B + x_D,$$

$$y_A + y_C = y_B + y_D.$$

Acestea sunt ecuatii liniare, deci grad 1.

Dreptunghi. Un paralelogram este dreptunghi daca doua laturi consecutive sunt perpendiculare:

$$\overrightarrow{AB} \cdot \overrightarrow{BC} = 0.$$

Adica:

$$(x_B - x_A)(x_C - x_B) + (y_B - y_A)(y_C - y_B) = 0.$$

Aceasta este ecuatie de grad 2.

13. Multimea $U(p) \cup D(p) \cup Int(p)$

Raspuns corect: (b) minim 1, maxim n .

Explicatie. In algoritmul de intersectii de segmente, un punct p din coada de evenimente apare deoarece este relevant pentru cel putin un segment: este capat superior, capat inferior sau punct de intersectie. Deci minimul este 1.

Maximul teoretic este n , daca toate cele n segmente au legatura cu acelasi punct p .

14. Dualitatea punctului $p = (4, -2)$

Raspuns corect: (a) Dreapta $y = 4x + 2$.

Formula dualitatii. Pentru un punct $p = (p_x, p_y)$:

$$p^* : y = p_x x - p_y.$$

Pentru $p = (4, -2)$:

$$p^* : y = 4x - (-2) = 4x + 2.$$

15. Programare liniara 2D - unic punct de maxim

Raspuns corect: (c) $f(x, y) = y$.

Semiplanele.

$$H_1 : -y - 2 \leq 0 \Rightarrow y \geq -2,$$

$$H_2 : -x - 2 \leq 0 \Rightarrow x \geq -2,$$

$$H_3 : x \leq 7,$$

$$H_4 : x + y \leq 10.$$

Regiunea fezabila este un patrulater. Daca maximizam y , cel mai sus punct este intersectia:

$$x = -2, \quad x + y = 10.$$

Deci:

$$y = 12.$$

Punctul unic de maxim este:

$$(-2, 12).$$

Pentru celelalte functii obiectiv, maximul se obtine pe o latura, deci nu este unic.

16. LPMARG2D

Raspuns corect: (d) $\mathcal{O}(n)$.

Explicatie. Algoritmul LPMARG2D parcurge semiplanele si actualizeaza solutia curenta. Daca pentru fiecare semiplan se ajunge la pasul 5, dar actualizarea marginala este facuta eficient, fiecare semiplan contribuie cu cost liniar total amortizat / constant pe iteratie in varianta discutata la curs. Pentru n semiplane:

$$T(n) = \mathcal{O}(n).$$

7 Intrebări deschise - rezolvari detaliate

17. Teorema Galeriei de Arta - amplasarea camerelor

Cerinta. Aplicati metoda din demonstratia teoremei galeriei de arta pentru poligonul $P_0P_1 \dots P_{12}$.

Teoria necesara. Teorema Galeriei de Arta spune ca orice poligon simplu cu n varfuri poate fi supravegheat cu cel mult:

$$\left\lfloor \frac{n}{3} \right\rfloor$$

camere.

Metoda standard:

1. triangulam poligonul;
2. 3-coloram varfurile triangulării;
3. alegem clasa de culoare cu cele mai putine varfuri;
4. punem camere in acele varfuri.

Fiecare triunghi are varfuri de toate cele trei culori, deci fiecare triunghi este vazut de o camera din clasa aleasa.

Aici avem $n = 13$, deci sunt suficiente:

$$\left\lfloor \frac{13}{3} \right\rfloor = 4$$

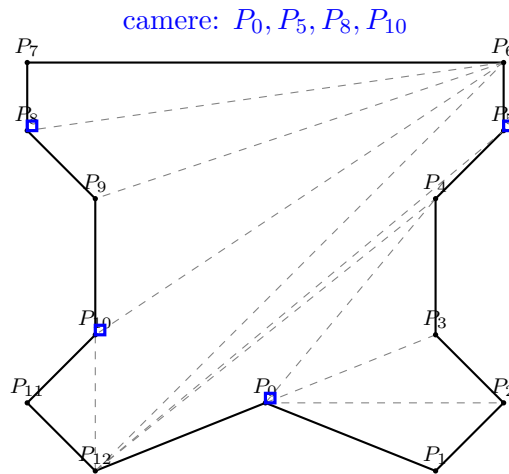
camere.

Coordonatele poligonului.

$$P_0 = (0, -4), P_1 = (5, -6), P_2 = (7, -4), P_3 = (5, -2), P_4 = (5, 2), P_5 = (7, 4), P_6 = (7, 6), \\ P_7 = (-7, 6), P_8 = (-7, 4), P_9 = (-5, 2), P_{10} = (-5, -2), P_{11} = (-7, -4), P_{12} = (-5, -6).$$

O posibilă triangulare si colorare. In desen, diagonalele sunt o triangulare valida, iar varfurile marcate cu patrat sunt o clasa de culoare minima. Camerele se pot pune in:

$$P_0, P_5, P_8, P_{10}.$$



Concluzie. O amplasare posibila este in P_0, P_5, P_8, P_{10} . Pot exista si alte raspunsuri corecte, daca triangularea si 3-colorarea sunt valide.

18. Triangulare cu 6 triunghiuri si 11 muchii

Cerinta. Realizati un desen cu o multime care admite o triangulare cu 6 triunghiuri si 11 muchii.

Formule. Pentru o multime de n puncte, dintre care k sunt pe frontiera acoperirii convexe:

$$T = 2n - k - 2,$$

$$M = 3n - k - 3.$$

Vrem:

$$2n - k - 2 = 6,$$

$$3n - k - 3 = 11.$$

Scadem prima ecuatie din a doua:

$$n - 1 = 5 \Rightarrow n = 6.$$

Apoi:

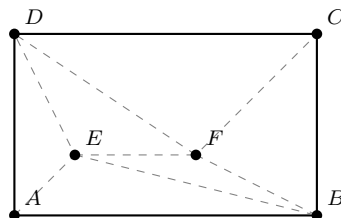
$$2 \cdot 6 - k - 2 = 6 \Rightarrow 10 - k = 6 \Rightarrow k = 4.$$

Deci avem nevoie de 6 puncte, dintre care 4 pe frontiera convexa si 2 in interior.

Exemplu.

$$M = \{(0, 0), (5, 0), (5, 3), (0, 3), (1, 1), (3, 1)\}.$$

Primele patru puncte sunt colturile dreptunghiului, ultimele doua sunt interioare.



Verificare.

$$n = 6, \quad k = 4,$$

$$T = 2 \cdot 6 - 4 - 2 = 6,$$

$$M = 3 \cdot 6 - 4 - 3 = 11.$$

19. Algoritmul lui Fortune - dificultate, structura auxiliara, relevanta

Cerinta. Explicati de ce line sweep direct este dificil, ce structura se foloseste si de ce este relevanta.

Problema. Vrem sa construim diagrama Voronoi. O idee naturala este sa folosim o dreapta de baleiere care coboara de sus in jos.

Dificultatea aplicarii directe. La un moment dat, o muchie sau un varf Voronoi poate depinde de situri care inca nu au fost intalnite de dreapta de baleiere. Deci nu putem decide direct doar din ce se afla deasupra liniei curente.

Structura auxiliara: beach line / curba parabolica. Fortune introduce **beach line**, adica frontiera dintre zona deja stabilita si zona care inca poate fi influentata de situri viitoare. Beach line este formata din arce de parabole.

De ce parabole? Pentru o pozitie a dreptei de baleiere l , punctele de pe beach line sunt la aceeasi distanta de:

- un sit deja descoperit;
- dreapta de baleiere.

Locul geometric al punctelor egal departate de un punct si o dreapta este o parabola.

Structuri mentinute.

- T = arbore de statut pentru arcele beach line;
- Q = coada de evenimente, cu evenimente de tip sit si evenimente de tip cerc;
- DCEL = structura in care construim progresiv diagrama Voronoi.

Relevanta. Ce este deasupra beach line nu mai poate fi modificat de evenimente viitoare, deci diagrama poate fi construita corect incremental.

20. DCEL - frontiere exterioare si interioare

Cerinta. Doua diferente intre frontiere exterioare si interioare.

DCEL. Doubly Connected Edge List este o structura pentru reprezentarea subdiviziunilor planare. Fiecare muchie geometrica este reprezentata prin doua semimuchii orientate, numite twin.

Diferenta 1: numar. O fata are exact o frontiera exterioara, dar poate avea zero, una sau mai multe frontiere interioare. Frontierele interioare apar cand fata are gauri.

Diferenta 2: orientare. In conventia uzuala:

- frontiera exterioara este parcursa astfel incat fata ramane la stanga;
- frontierele interioare sunt parcurse invers, astfel incat gaura sa ramana pe partea corecta.

De aceea, virajele in varfuri convexe apar diferit: la stanga pentru frontiera exterioara si la dreapta pentru frontiera interioara.

21. Overlay DCEL pentru doua patrate

Cerinta. $ABCD$ este patrat, iar M, N, P, Q sunt mijloacele laturilor. Consideram subdiviziunile planare asociate patratelor $ABCD$ si $MNPQ$. Cate varfuri, semimuchii si fete are overlay-ul?

Raspuns corect: 8 varfuri, 24 semimuchii, 6 fete.

Explicatie. Varfurile overlay-ului sunt:

$$A, B, C, D, M, N, P, Q,$$

deci:

$$V = 8.$$

Pe frontiera patratului mare, fiecare latura este impartita in doua de mijlocul ei. Deci frontiera externa are:

$$8$$

muchii geometrice. Patrutul interior $MNPQ$ mai are:

$$4$$

muchii geometrice. Total:

$$E = 8 + 4 = 12.$$

In DCEL fiecare muchie geometrica are doua semimuchii:

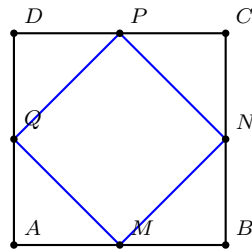
$$H = 2E = 24.$$

Prin formula lui Euler pentru graf planar conex:

$$V - E + F = 2.$$

Deci:

$$8 - 12 + F = 2 \Rightarrow F = 6.$$



22. Dualitatea configuratiei cu patru drepte concurente

Cerinta. Determinati duala configuratiei: patru drepte trec prin acelasi punct M ; pe doua dintre ele alegem cate un punct diferit de M ; dreapta prin cele doua puncte.

Regula de dualitate.

- Punctul $p = (p_x, p_y)$ devine dreapta:

$$p^* : y = p_x x - p_y.$$

- Dreapta neverticala $d : y = mx + n$ devine punctul:

$$d^* = (m, -n).$$

Ce se intampla cu incidentele? Daca un punct este pe o dreapta in primal, atunci dreapta duala a punctului trece prin punctul dual al dreptei.

Configuratia initiala. Patru drepte concurente in M devin patru puncte coliniare pe dreapta M^* .

Cele doua puncte alese. Fie punctele alese A si B , fiecare pe cate una dintre doua drepte prin M . In dual:

- A devine dreapta A^* ;
- B devine dreapta B^* ;

- pentru ca A era pe una dintre dreptele initiale, A^* trece prin punctul dual al acelei drepte;
- la fel pentru B .

Dreapta AB din primal devine punctul $(AB)^*$, care este intersectia dreptelor A^* si B^* .

Duala descrisa in cuvinte. Patru puncte coliniare pe o dreapta d ; alegem doua dintre ele; prin fiecare trasam cate o dreapta diferita de d ; intersectia acestor doua drepte este punctul dual al dreptei determinate de punctele initiale.

23. De ce se omit dreptele verticale in dualitate

Cerinta. De ce in definirea dualitatii sunt omise dreptele verticale?

Raspuns. Formula folosita in curs asociaza dreptei:

$$d : y = mx + n$$

punctul:

$$d^* = (m, -n).$$

Aceasta formula foloseste panta m a dreptei.

Pentru o dreapta verticala:

$$x = a,$$

panta nu este definita. Intuitiv, panta ar fi infinita, deci nu putem construi un punct dual finit de forma $(m, -n)$.

Concluzie. Dreptele verticale sunt omise deoarece nu pot fi scrise sub forma $y = mx + n$ si nu au panta finita.

24. Testul de orientare - complexitate algebrica si aplicatii

Cerinta. Care este complexitatea algebrica a testului de orientare? Indicati trei teme unde apare.

Formula. Pentru punctele:

$$P = (p_x, p_y), \quad Q = (q_x, q_y), \quad R = (r_x, r_y),$$

testul de orientare este:

$$\Delta(P, Q, R) = \begin{vmatrix} 1 & 1 & 1 \\ p_x & q_x & r_x \\ p_y & q_y & r_y \end{vmatrix}.$$

Echivalent:

$$\Delta(P, Q, R) = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x).$$

Acesta este un polinom de gradul 2, deci complexitatea algebrica este grad 2.

Interpretare.

$$\Delta > 0 \Rightarrow R \text{ este la stanga lui } \overrightarrow{PQ},$$

$$\Delta < 0 \Rightarrow R \text{ este la dreapta lui } \overrightarrow{PQ},$$

$$\Delta = 0 \Rightarrow P, Q, R \text{ sunt coliniare.}$$

Aplicatia 1: Graham's scan / acoperire convexa. Cand adaugam un punct in lista, verificam ultimele trei puncte. Daca obtinem viraj la dreapta, punctul din mijloc este eliminat. Context:

$$\Delta(P_{k-1}, P_k, P_{nou}) \leq 0 \Rightarrow \text{eliminam } P_k$$

(in functie de conventia pentru marginea superioara/inferioara si daca pastram coliniarele).

Aplicatia 2: Jarvis' march. Cand cautam urmatorul varf al acoperirii convexe, comparam candidatii prin orientare. Daca un punct este mai la dreapta / stanga decat pivotul curent, actualizam pivotul.

Aplicatia 3: triangularea poligoanelor monotone. Testul de orientare decide daca o diagonala este interioara si daca un varf produce cazul corect in algoritmul cu stiva.

Alte aplicatii posibile.

- intersectia segmentelor;
- clasificarea varfurilor convexe/concave;
- determinarea pozitiei unui punct fata de o muchie orientata;
- verificari locale in triangularea Delaunay.

8 Ce ar mai putea sa dea ca teorie la Partea II

1. Testul de orientare: formula, semn, complexitate algebrica.
2. Coliniaritate in $\mathbb{R}^2$ si $\mathbb{R}^3$ prin vectori proportionali.
3. Graham's scan: sortare, stiva/lista, eliminari la viraje gresite, $\mathcal{O}(n \log n)$.
4. Jarvis' march: gift wrapping, $\mathcal{O}(nh)$, cand este mai bun decat Graham.
5. Clasificarea varfurilor: convex/concav si tipuri pentru triangulare.
6. Ear clipping si Teorema Galeriei de Arta.
7. Triangularea poligoanelor monotone: lanturi, stiva, cazuri.
8. Formulele pentru triangularea multimilor de puncte:

$$T = 2n - k - 2, \quad E = 3n - k - 3.$$

9. Graf planar: formula Euler $v - m + f = 2$ si consecinte.
10. Diagrama Voronoi: semidrepte = puncte pe hull, limite $n_v \leq 2n - 5$, $n_m \leq 3n - 6$.
11. Fortune: site events, circle events, beach line, status tree, event queue.
12. DCEL: vertices, half-edges, faces, twin, next, prev, incident face.
13. Overlay de subdiviziuni: numarare prin Euler si impartirea muchiilor.
14. Dualitate: punct $\leftrightarrow$ dreapta, incidente se pastreaza invers.
15. Semiplane si programare liniara 2D: intersectie vida, punct, segment, poligon, nemarginita.